UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Mini Project Report

On

Stylist Salon: Online Appointment Booking System

(Distributed Database Management System mini project, MySQL)

SUBMITTED BY:

ISHITA THAKUR
UID: 26MCI10169
BRANCH: MAM-3
SECTION / GROUP: B
SEMESTER: I

SUBMITTED TO:

Mr. Chanchal Sharma
SUBJECT: PL/SQL
SUBJECT CODE: 26CAP-602
DATE: 28-10-2026

Table of Content

S. No.	Name	Page no.
1	Introduction / Background	3
2	Objectives / Technologies Used	3
3	Requirements and System Design	3
4	Source Code & Output	6
5	Functionalities	14
6	Conclusion / Future Work / Learning Outcomes	15

1. Introduction

This report describes **Stylist Salon**, an online appointment booking system for a salon chain with three branches: **Downtown, Campus Road and Mall Road**. Customers register, search services, fill an appointment cart and book a stylist for a date and time slot. Each booking is stored in its own branch's table, and the same stylist can never be booked twice for the same date and slot. The project is a **Distributed Database Management System (DDBMS)** mini project built in MySQL.

2. Background

A salon chain has three problems: many online bookings can clash, each branch mostly uses its own data, and the owner still wants one combined report. A distributed database solves this. Appointment data is split by branch (fragmentation) and kept at that branch (allocation), shared data such as the service menu is copied everywhere (replication), and a global view shows everything as one table. Here the three "sites" are three tables on one MySQL 8.0 server; the design is the same as for separate machines.

3. Objectives

- User registration, login, profile update; admin service and staff management; search by name, category and price.
- Appointment cart, checkout, history, status flow, cancel and reschedule; reports for services, hours, stylists and revenue.
- Demonstrate: fragmentation, replication, data dictionary, query processing, ACID, locking, deadlock, recovery, triggers, roles and 3NF.

4. Technologies Used

- **Database:** MySQL 8.0.43 (InnoDB) with stored procedures, functions, triggers, views, roles and CHECK constraints; MySQL Workbench and command-line client.
- **Front end:** HTML5, CSS3 and JavaScript single page that simulates the database logic in the browser.
- **Automation:** PowerShell scripts that build the database and run every demo, including two simultaneous sessions for locking tests.

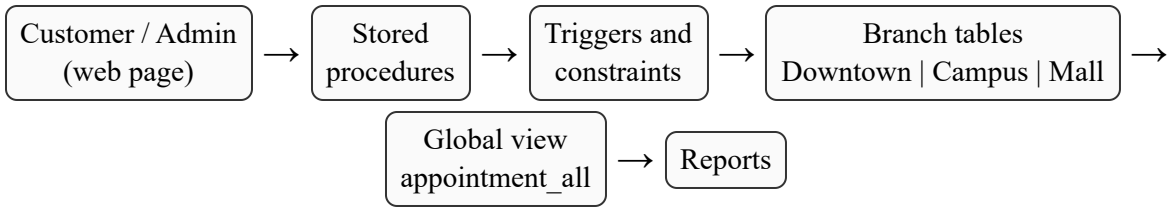
5. Requirements

- MySQL Server 8.0.16 or later (CHECK constraints) and MySQL Workbench or the command-line client.
- PowerShell (to run *run.ps1*) and a modern web browser for *website/index.html*.

6. System Design

Architecture Overview

Every action in the web page becomes a call to a stored procedure. The procedure checks the rules, saves data inside a transaction, and writes to the correct branch table. Reports read the global view.



Database Life Cycle (DBLC)

Phase	What was done
Requirements	Six features: users, services and staff, cart, appointments, reports, security.
Conceptual / logical	ER diagram, then tables normalized to 3NF.
Distributed design	Fragments per branch, allocation, replicated menu, data dictionary.
Physical / implementation	InnoDB tables, keys, UNIQUE slot index; schema, triggers, procedures, data (files 01 to 03).
Testing / maintenance	Good and bad test cases (file 04), concurrency tests, roles and users (file 05).

ER Diagram and Normalization (3NF)

```
BRANCH 1 ----< STYLIST >----< STYLIST_SERVICE >---- SERVICE
   |            |                                    |
   +--< CUSTOMER  +---< APPOINTMENT >---< APPOINTMENT_ITEM >-+     APPOINTMENT 1 ---- 1 PAYMENT
          +--< CART_ITEM >--- SERVICE
```

- **1NF:** repeating groups (service 1, service 2) moved to rows of *appointment_item*.
- **2NF:** service name and category depend only on *service_id*, so they live in *service*.
- **3NF:** customer, branch and stylist facts are stored once each in their own table. The price is copied into *appointment_item* on purpose, to keep the price paid at booking time.

Tables: branch, time_slot, service, stylist, stylist_service, customer (salted hash, role), cart_item, appointment_downtown / _campus / _mall, appointment_item, payment, cancel_log, service_replica, data_dictionary.

Distributed Database Design

Horizontal fragmentation: the appointment table is cut by rows into one fragment per branch. A trigger enforces the rule so a row cannot enter the wrong fragment.

Fragment	Rule	Allocation (stored at)

appointment_downtown	stylist.branch_id = 1	Downtown site
appointment_campus	stylist.branch_id = 2	Campus Road site
appointment_mall	stylist.branch_id = 3	Mall Road site

Replication: service, branch and time_slot are copied to every site; *service_replica* holds one copy per site and triggers keep the copies equal to the master. **Data dictionary:** table *data_dictionary* lists each table, its fragment type, rule and site. **Transparency:** the view *appointment_all* (UNION ALL) hides the split. Type: homogeneous DDBMS (all sites MySQL).

Query Processing

A global question such as "total revenue of all branches" is split into one small query per branch (each runs at one site) and the answers are added (Fig. 3). A question about one branch reads only that fragment.

Transactions, ACID, Locking, Deadlock and Recovery

Checkout is one transaction: appointment, services, payment, empty cart. Any error means ROLLBACK and nothing is saved; success means COMMIT.

Atomicity	checkout rolls back on any error; ROLLBACK and SAVEPOINT demo
Consistency	CHECK, FOREIGN KEY, UNIQUE and trigger rules refuse bad data
Isolation	InnoDB row locks: a second session waits; two users cannot take one slot
Durability	redo log flushed at every COMMIT (write-ahead logging)

No double booking: *active_key* (stylist|date|slot) is UNIQUE; cancelling sets it to NULL, which frees the slot. **Deadlock:** A holds service 2 and wants 3, B holds 3 and wants 2; MySQL detects the cycle and rolls one session back (ERROR 1213). Prevention: lock in the same order, keep transactions short.

Write-Ahead Logging (WAL): the log record is written to disk before the data page. After a crash, committed work is replayed from the log (REDO) and unfinished work is undone (UNDO). In the demo the log counter *Innodb_os_log_written* rose from 955392 to 957440 bytes after one COMMIT, with *innodb_flush_log_at_trx_commit = 1*.

Triggers and Security

- **Triggers:** before insert (block past dates, block a stylist of another branch), before update (no rescheduling into the past), after update (write *cancel_log* when cancelled), and two on *service* that copy changes to every site.
- **Passwords:** SHA-256 of (random salt + password); the real password is never stored.
- **Bind variables:** every procedure uses parameters, no SQL is built from user text, so injection fails (*x' OR '1'='1* returns 0).
- **Roles:** *salon_customer* (menu, cart, book, cancel, reschedule), *salon_stylist* (view appointments, change status), *salon_admin* (everything). Only admin touches tables directly; others get

EXECUTE on specific procedures (Fig. 5).

Case Study: Middleware over Different Databases

Downtown keeps an old system (APPT_NO, text dates like 28-OCT-2026, money in paise); Campus Road keeps a newer app (id, DATETIME, rupees). A middleware layer gives one **integrated view**:

Middleware part	Job and salon example
Schema mapping	APPT_NO and id both become appt_ref
Query translation	one global query rewritten per source: APPT_DT = '29-OCT-2026' for Downtown, DATE(starts_at) = '2026-10-29' for Campus
Data conversion	text date to DATE, paise / 100 to rupees
Source selection	a Campus-only question never touches Downtown
Result integration	UNION ALL of the mapped views = integrated_appointments

Benefit: one global view and local freedom for each branch. Cost: an extra layer, slower queries, and cross-system transactions need two-phase commit. Demo: *06_middleware_demo.sql*.

7. Source Code & Output

Only the main code is shown for explanation; the full code is in *database/01_schema.sql* to *07_concurrency_demo.ps1*.

Database code: fragments and global view

```
-- Horizontal fragment: one table per branch (same shape for Campus and Mall)
CREATE TABLE appointment_downtown (
  appt_id INT PRIMARY KEY, customer_id INT NOT NULL, stylist_id INT NOT NULL,
  appt_date DATE NOT NULL, slot CHAR(5) NOT NULL,
  status ENUM('Booked','In Progress','Completed','Cancelled') NOT NULL DEFAULT 'Booked',
  total DECIMAL(8,2) NOT NULL,
  -- "stylist|date|slot" while alive, NULL when cancelled -> UNIQUE = no double booking
  active_key VARCHAR(40) GENERATED ALWAYS AS
    (IF(status='Cancelled', NULL, CONCAT(stylist_id,'|',appt_date,'|',slot))) STORED,
  UNIQUE KEY ux_slot (active_key), ... FOREIGN KEYs ...);

-- Global view: the 3 fragments look like ONE table
CREATE VIEW appointment_all AS
  SELECT 1 AS branch_id, appt_id, customer_id, stylist_id, appt_date, slot, status, total FROM
appointment_downtown
  UNION ALL SELECT 2, appt_id, customer_id, stylist_id, appt_date, slot, status, total FROM
appointment_campus
  UNION ALL SELECT 3, appt_id, customer_id, stylist_id, appt_date, slot, status, total FROM
appointment_mall;
```

Database code: triggers

```
CREATE TRIGGER trg_bi_downtown BEFORE INSERT ON appointment_downtown FOR EACH ROW
BEGIN
  IF NEW.appt_date < CURDATE() AND @seed_mode IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot book a past date';
  END IF;
  IF (SELECT branch_id FROM stylist WHERE stylist_id = NEW.stylist_id) <> 1 THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Fragment rule: stylist not in Downtown';
  END IF;
END;
CREATE TRIGGER trg_au_downtown AFTER UPDATE ON appointment_downtown FOR EACH ROW
BEGIN
  IF NEW.status = 'Cancelled' AND OLD.status <> 'Cancelled' THEN
    INSERT INTO cancel_log (appt_id, cancelled_on, branch_name) VALUES (OLD.appt_id, NOW(),
'Downtown');
  END IF;
END;
```

Database code: checkout transaction

```
CREATE PROCEDURE checkout(IN p_customer INT, IN p_branch INT, IN p_stylist INT,
                          IN p_date DATE, IN p_slot CHAR(5), OUT p_msg VARCHAR(200))
BEGIN
  ... DECLARE variables ...
  DECLARE EXIT HANDLER FOR 1062            -- UNIQUE slot key hit = slot already taken
  BEGIN ROLLBACK; SET p_msg = 'ROLLBACK: that stylist is already booked ...'; END;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION    -- any other error (past date, wrong branch ...)
  BEGIN GET DIAGNOSTICS CONDITION 1 v_err = MESSAGE_TEXT; ROLLBACK;
        SET p_msg = CONCAT('ROLLBACK: ', v_err, '. Nothing saved.'); END;

  START TRANSACTION;
  ... checks: cart not empty, date not in the past,
      stylist works at this branch and does every service in the cart ...
  INSERT INTO appt_seq VALUES (NULL);  SET v_id = LAST_INSERT_ID();
  IF p_branch = 1 THEN INSERT INTO appointment_downtown (...) VALUES (...);   -- right fragment
  ELSEIF p_branch = 2 THEN INSERT INTO appointment_campus (...) VALUES (...);
  ELSEIF p_branch = 3 THEN INSERT INTO appointment_mall (...) VALUES (...);  END IF;
  INSERT INTO appointment_item SELECT v_id, service_id, price FROM v_cart WHERE customer_id =
p_customer;
  INSERT INTO payment (appt_id, amount) VALUES (v_id, v_total);
  DELETE FROM cart_item WHERE customer_id = p_customer;
  COMMIT;
  SET p_msg = CONCAT('COMMIT: appointment ', v_id, ' booked ...');
END
```

Database code: query processing

```
-- global query split into one small query per branch, answers added (WITH ROLLUP = total row)
SELECT IF(GROUPING(site),'GLOBAL TOTAL',site) AS site, SUM(revenue) AS revenue FROM (
  SELECT 'Downtown'    AS site, COALESCE(SUM(total),0) AS revenue FROM appointment_downtown WHERE
status<>'Cancelled'
  UNION ALL SELECT 'Campus Road', COALESCE(SUM(total),0) FROM appointment_campus WHERE
status<>'Cancelled'
  UNION ALL SELECT 'Mall Road',   COALESCE(SUM(total),0) FROM appointment_mall   WHERE
status<>'Cancelled'
) AS branch_answers GROUP BY site WITH ROLLUP;
```

Database code: registration and login

```
-- salted SHA-256 hash, the real password is never stored
INSERT INTO customer (name, phone, email, address, salt, password_hash, home_branch)
VALUES (p_name, p_phone, p_email, p_address, v_salt, SHA2(CONCAT(v_salt, p_password), 256),
p_branch);

CREATE FUNCTION login_customer(p_email VARCHAR(60), p_password VARCHAR(100)) RETURNS INT READS SQL
DATA
```

```
BEGIN
  DECLARE v_id INT DEFAULT 0;
  SELECT customer_id INTO v_id FROM customer
  WHERE email = p_email AND password_hash = SHA2(CONCAT(salt, p_password), 256) LIMIT 1;
  RETURN v_id;                                    -- 0 = wrong email or password
END
```

Database code: roles

```
CREATE ROLE 'salon_customer', 'salon_stylist', 'salon_admin';
GRANT SELECT  ON stylist_salon.v_service_menu       TO 'salon_customer';
GRANT EXECUTE ON PROCEDURE stylist_salon.checkout   TO 'salon_customer';
GRANT EXECUTE ON PROCEDURE stylist_salon.cancel_appointment TO 'salon_customer';
GRANT EXECUTE ON PROCEDURE stylist_salon.set_status TO 'salon_stylist';
GRANT ALL PRIVILEGES ON stylist_salon.*             TO 'salon_admin';
```

Front-End code

```
function book(){            // website: simulates the checkout transaction
 ... checks login, cart, branch, stylist, date, slot ...
 log("BEGIN transaction");
 if(taken(+f.st,f.date,f.slot)){          // same stylist + date + slot already booked
   log("INSERT ... blocked: trigger fired"); log("ROLLBACK: nothing was saved");
   S.msg={k:"er",t:"That slot was just taken. The booking was rolled back."}; return draw()}

S.appts.push({id:id,cu:S.user.name,b:f.branch,st:+f.st,d:f.date,sl:f.slot,it:S.cart.slice(),s:"Booked
"});
 log("INSERT INTO payment (...)"); log("COMMIT: booking saved"); ... }
```

Output

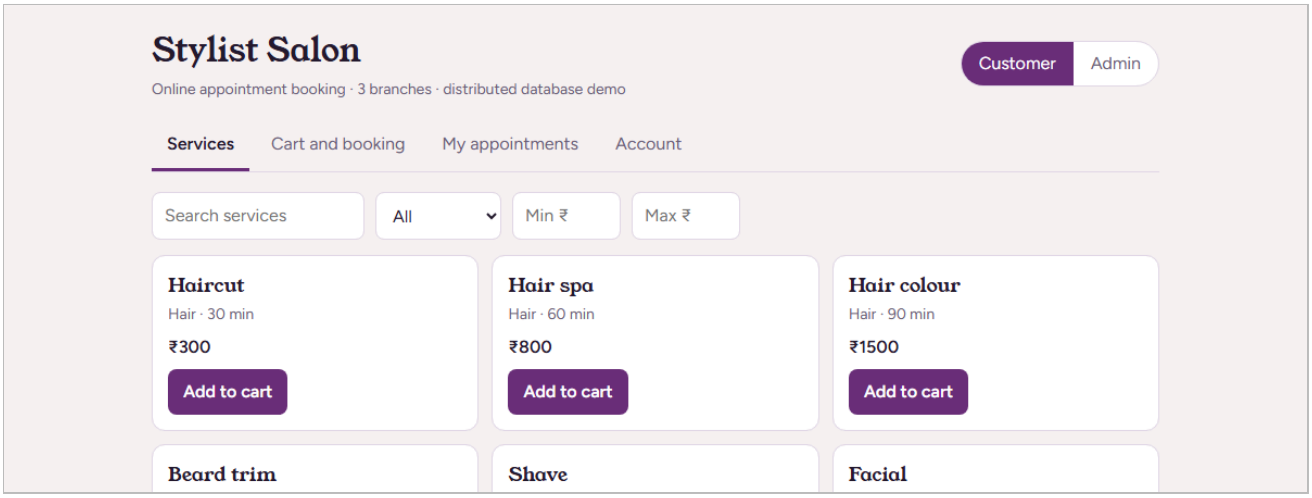


Fig 1. Website: service menu with search, category and price filters

Checkout = one transaction: COMMIT or ROLLBACK

```
+-----------------------------------------------------------------+
| booking_1                                                       |
+-----------------------------------------------------------------+
| COMMIT: appointment 14 booked for 2026-10-29 11:00, total Rs 450.00 |
+-----------------------------------------------------------------+

+-------------------------------------------------------------------------+
| booking_2_clash                                                         |
+-------------------------------------------------------------------------+
| ROLLBACK: that stylist is already booked at that date and time. Nothing saved. |
+-------------------------------------------------------------------------+

+-------------------------------------------------+
| past_date_result                                |
+-------------------------------------------------+
| ROLLBACK: Cannot book a past date. Nothing saved. |
+-------------------------------------------------+

+-----------------------------------------------------------------------------------------------+
| wrong_branch_result                                                                           |
+-----------------------------------------------------------------------------------------------+
| ROLLBACK: Stylist is not at this branch or does not do every service in the cart. Nothing saved. |
+-----------------------------------------------------------------------------------------------+

ERROR 1644 (45000) Fragment rule: this stylist does not belong to Campus Road
```

Fig 2. Checkout: COMMIT for a free slot, ROLLBACK for a clash, past date and wrong branch

Query processing: branch answers then global total

```
+-------------+---------+
| site        | revenue |
+-------------+---------+
| Downtown    | 2870.00 |
| Campus Road | 3700.00 |
| Mall Road   | 4350.00 |
+-------------+---------+

+----------------------------+
| total_revenue_all_branches |
+----------------------------+
|                   10920.00 |
+----------------------------+

+--------------+----------+
| site         | revenue  |
+--------------+----------+
| Campus Road  |  3700.00 |
| Downtown     |  2870.00 |
| Mall Road    |  4350.00 |
| GLOBAL TOTAL | 10920.00 |
+--------------+----------+
```

Fig 3. Query processing: branch answers, then the global total

Fragmentation: rows per branch + data dictionary

```
+-------------+-------------+
| site        | rows_stored |
+-------------+-------------+
| Downtown    |           7 |
| Campus Road |           5 |
| Mall Road   |           5 |
+-------------+-------------+

+--------------------------+---------------------------------+-------------------------+-------------------------------+
| table_name               | fragment_type                   | fragmentation_rule      | stored_at                     |
+--------------------------+---------------------------------+-------------------------+-------------------------------+
| appointment_downtown     | Horizontal fragment             | stylist.branch_id = 1   | Downtown site only            |
| appointment_campus       | Horizontal fragment             | stylist.branch_id = 2   | Campus Road site only         |
| appointment_mall         | Horizontal fragment             | stylist.branch_id = 3   | Mall Road site only           |
| appointment_all          | Global view (UNION ALL)         | union of the 3 fragments | Any site (virtual)           |
| service                  | Replicated                      | full copy at every site | All 3 sites (service_replica) |
| branch / time_slot       | Replicated                      | full copy at every site | All 3 sites                   |
| stylist                  | Allocated by branch_id          | stylist.branch_id       | Own branch site               |
| customer                 | Logical split by home_branch    | customer.home_branch    | Home branch site              |
| payment, appointment_item | Allocated with their appointment | appt_id                | Same site as the appointment  |
+--------------------------+---------------------------------+-------------------------+-------------------------------+
```

Fig 4. Fragmentation: rows stored per branch and the data dictionary

Security test: u_customer (role salon_customer)

```
+----------------------+----------------------+
| logged_in_as         | active_role          |
+----------------------+----------------------+
| u_customer@localhost | `salon_customer`@`%` |
+----------------------+----------------------+

+-------------+
| login_works |
+-------------+
|           1 |
+-------------+

ERROR 1142 (42000) SELECT command denied to user 'u_customer'@'localhost' for table 'customer'

ERROR 1142 (42000) SELECT command denied to user 'u_customer'@'localhost' for table 'service'

ERROR 1142 (42000) DELETE command denied to user 'u_customer'@'localhost' for table 'service'

ERROR 1142 (42000) UPDATE command denied to user 'u_customer'@'localhost' for table 'payment'

ERROR 1370 (42000) execute command denied to user 'u_customer'@'localhost' for routine

ERROR 1370 (42000) execute command denied to user 'u_customer'@'localhost' for routine
```

Fig 5. Security: customer role is denied direct table access



Fig 6. Write-ahead log: log is flushed at every COMMIT and the bytes written rise

Concurrency: two users race for one slot (B waits, then is rolled back)

```
----- C2_sessionA -----

+-------------------------------------------------+-------------------------+
| step                                            | at_time                 |
+-------------------------------------------------+-------------------------+
| A: books Aman, tomorrow 17:00 (not committed yet) | 2026-10-28 07:08:37.458 |
+-------------------------------------------------+-------------------------+

+-----------+-------------------------+
| step      | at_time                 |
+-----------+-------------------------+
| A: COMMIT | 2026-10-28 07:08:41.462 |
+-----------+-------------------------+

----- C2_sessionB -----

+-----------------------------------------------------+-------------------------+
| step                                                | at_time                 |
+-----------------------------------------------------+-------------------------+
| B: tries the SAME stylist, date and slot (waits for A) | 2026-10-28 07:08:38.669 |
+-----------------------------------------------------+-------------------------+

+-----------------------------------------------------------------------------------+-------------------------+
| B_result                                                                          | at_time                 |
+-----------------------------------------------------------------------------------+-------------------------+
| ROLLBACK: that stylist is already booked at that date and time. Nothing saved. | 2026-10-28 07:08:41.469 |
+-----------------------------------------------------------------------------------+-------------------------+
```

Fig 7. Concurrency: two users race for one slot; the second is rolled back

Deadlock: MySQL picks a victim (ERROR 1213)

```
----- C3_sessionA -----

+--------------------+-------------------------+
| step               | at_time                 |
+--------------------+-------------------------+
| A: locks service 2 | 2026-10-28 07:08:41.618 |
+--------------------+-------------------------+

+-----------------------------------+-------------------------+
| step                              | at_time                 |
+-----------------------------------+-------------------------+
| A: now wants service 3 (B holds it) | 2026-10-28 07:08:44.628 |
+-----------------------------------+-------------------------+

+-------------------------------------------+
| step                                      |
+-------------------------------------------+
| A: next statement ran (A was not the victim) |
+-------------------------------------------+

----- C3_sessionB -----

+--------------------+-------------------------+
| step               | at_time                 |
+--------------------+-------------------------+
| B: locks service 3 | 2026-10-28 07:08:42.668 |
+--------------------+-------------------------+

+-----------------------------------+-------------------------+
| step                              | at_time                 |
+-----------------------------------+-------------------------+
| B: now wants service 2 (A holds it) | 2026-10-28 07:08:45.681 |
+-----------------------------------+-------------------------+

+-----------------------------------------------------------------------------------------+
| step                                                                                    |
+-----------------------------------------------------------------------------------------+
| B: this line only prints because of --force; B was already rolled back by the deadlock |
+-----------------------------------------------------------------------------------------+

ERROR 1213 (40001) Deadlock found when trying to get lock; try restarting transaction
```

Fig 8. Deadlock: MySQL rolls one session back (ERROR 1213)

8. Functionalities

- **Users:** registration, login (salted hash), profile update. **Services and staff:** admin add, update, remove; search by name, category and price range; add stylists and assign services.

- **Cart and appointments:** add, update and remove services; checkout with branch, stylist, date and slot; history; Booked, In Progress, Completed, Cancelled; cancel and reschedule.
- **Reports:** most booked service, busiest hours, stylist-wise bookings, daily and monthly revenue, revenue per branch.
- **Distributed and security:** fragments, global view, replication, data dictionary, locking, deadlock handling, write-ahead log, roles and bind variables.

9. Conclusion

Stylist Salon is a working distributed booking database. Appointments are fragmented by branch yet read as one table, the menu is replicated, and every booking is all-or-nothing. Tests showed that double booking, past dates, wrong-branch bookings and bad data are rejected, locks and deadlocks are handled, and each role does only its own job.

10. Future Work

- Place the fragments on separate MySQL servers (FEDERATED tables or replication) with two-phase commit.
- Connect the web page to the database through an API; add online payment and reminders.
- Stronger hashing (bcrypt or Argon2), HTTPS and regular encrypted backups.

11. Learning Outcomes

- Designing a 3NF schema and following the database life cycle.
- Fragmentation, allocation, replication, data dictionary and global views; splitting a global query.
- Transactions (COMMIT, ROLLBACK, SAVEPOINT), ACID, locking, deadlock and write-ahead logging.
- Triggers, stored procedures, roles and bind variables; middleware integration of different databases.

12. GitHub link:

Not uploaded (full source code is in the project folder: database/, website/, docs/).

13. Bibliography:

- MySQL. (n.d.). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/
- MySQL. (n.d.). InnoDB redo log. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/innodb-redo-log.html
- Elmasri, R. and Navathe, S. B. Fundamentals of Database Systems. Pearson.